Patrones de Diseño

Iterator

Propósito

Iterator es un patrón de diseño de **comportamiento** que permite recorrer los elementos de una colección de objetos sin exponer su representación o implementación interna.

Contexto

Las colecciones son unas de los tipos de datos más utilizados en programación. Sin embargo, una colección no es más que un contenedor de objetos. Varias colecciones almacenan sus elementos en listas o arreglos, sin embargo otras se basan en pilas, árboles, grafos u otras estructuras de datos complejas.

Sin importar cómo se almacenan los elementos, va a ser necesario en algún momento acceder a los mismos para utilizarlos “fuera” de la colección

Problema

¿Podría implementar un método que sea capaz de recorrer un *contenedor* de objetos independientemente de cómo estén almacenados los mismos?

¿Qué métodos necesitaría para crear un bucle que recorra los elementos?

Si fuera un ArrayList...

```
public void imprimirElementos(ArrayList contenedor){
    int i = 0;
    while (i<contenedor.size()) {
        System.out.println(contenedor.get(i));
        i++;
    }
}
```

Y si no?...

```
public void imprimirElementos(?????? contenedor){
    contenedor.irAlPrimero();
    while (contenedor.tieneMas()) {
        System.out.println(contenedor.siguiente());
        contenedor.avanzar();
    }
}
```

Los “contenedores” deberían saber responder a los siguientes mensajes:

- irAlPrimero()
- tieneMas ()
- siguiente()
- avanzar()

```
public void imprimirElementos(?????? contenedor){
    contenedor.irAlPrimero();
    while (contenedor.tieneMas()) {
        System.out.println(contenedor.siguiente());
        contenedor.avanzar();
    }
}
```

¿De qué tipo declaro entonces el parámetro del método *imprimirElementos*?

Solución

El patrón Iterator sugiere que definamos el comportamiento necesario para recorrer un contenedor en una interface *Iterador* y que luego creemos objetos encargados de recorrer la estructura.

Separar del contenedor la implementación de los métodos encargados de recorrerlo

Estructura del patrón Iterator

Lo que quiero recorrer

está en java

«interface»
Iterable

Crea clases, declara uno o mas metodos para obtener los iteradores

+ iterator():Iterator

Cliente

«interface»
Iterator

declara las operaciones necesarias para recorrer el contenedor

+ hasNext():boolean

+ next():Object

Esto es lo que creo que es el iterador

Implementa

implementa

My SimpleLinkedList

IterableConcreto

- elementos

+ ...

+ iterator():Iterator

crea

IteratorConcreto

- acceso_a_los_elementos

- estado_iteracion

+ hasNext():boolean

+ next():Object

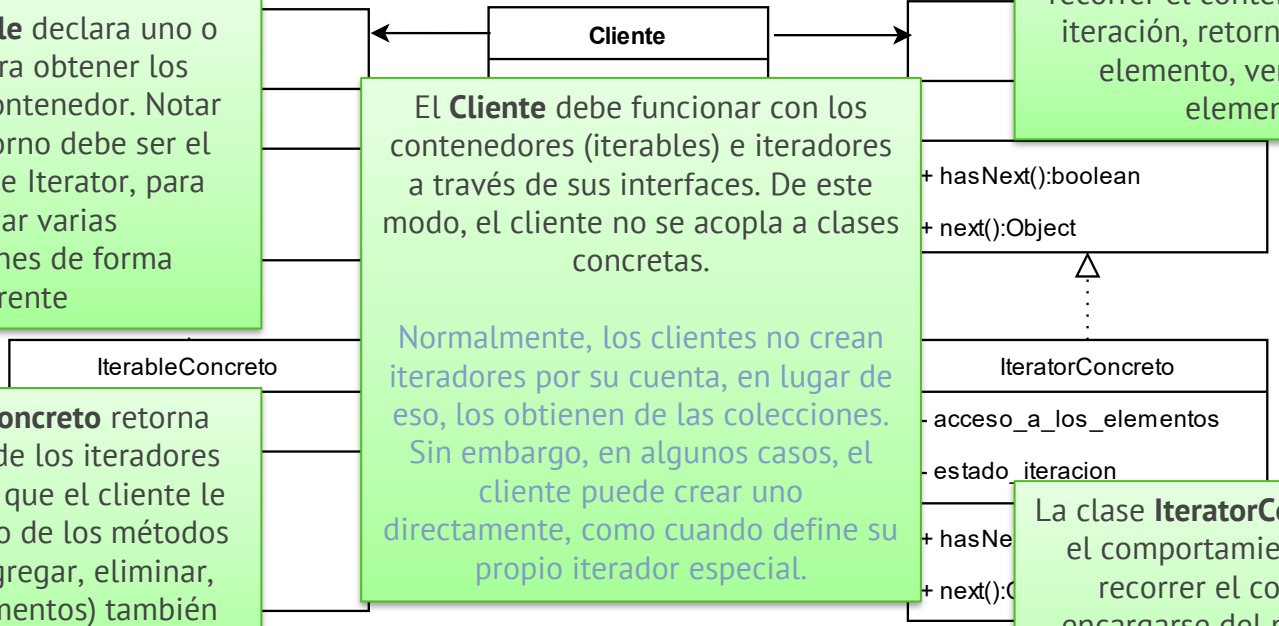
En el constructor me aseguro que se pare en el primer elemento

```
Iterator<T>it=lista.iterator()
While(it.hasNext())
T info = it.next()
//hago lo que quiera
```

Estructura del patrón Iterator

La interface **Iterable** declara uno o más métodos para obtener los iteradores de un contenedor. Notar que el tipo de retorno debe ser el tipo de la interface Iterator, para poder utilizar varias implementaciones de forma transparente

La interface **Iterator** declara las operaciones necesarias para recorrer el contenedor: iniciar la iteración, retornar el siguiente elemento, ver si hay más elementos...



El **Cliente** debe funcionar con los contenedores (iterables) e iteradores a través de sus interfaces. De este modo, el cliente no se acopla a clases concretas.

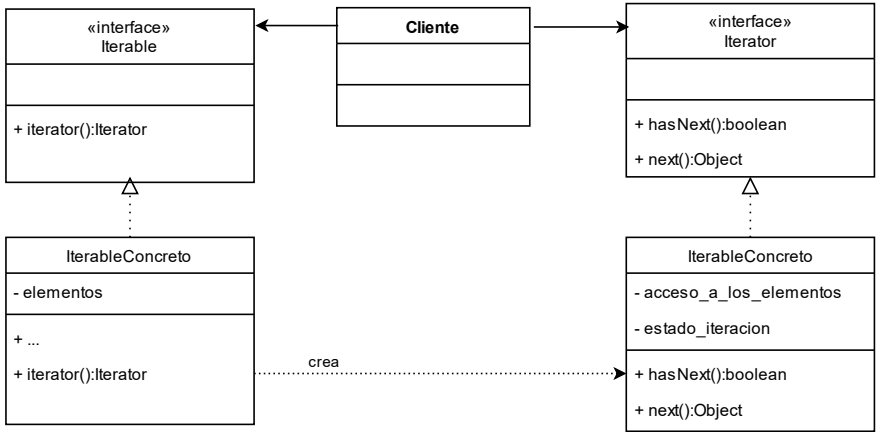
Normalmente, los clientes no crean iteradores por su cuenta, en lugar de eso, los obtienen de las colecciones. Sin embargo, en algunos casos, el cliente puede crear uno directamente, como cuando define su propio iterador especial.

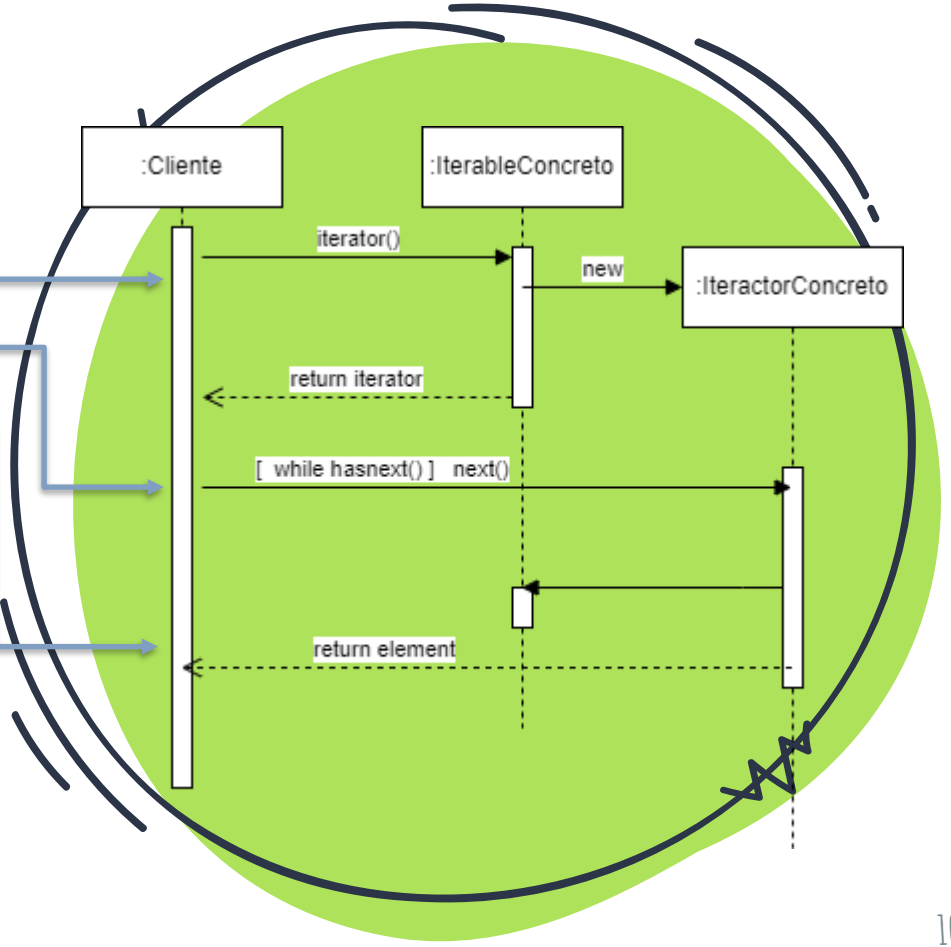
La clase **IterableConcreto** retorna nuevas instancias de los iteradores concretos cada vez que el cliente le solicita uno. El resto de los métodos del contenedor (agregar, eliminar, consultar, etc. elementos) también está en esta clase.

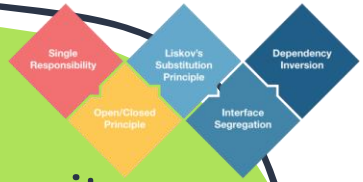
La clase **IteratorConcreto** Implementa el comportamiento concreto para recorrer el contenedor. Debe encargarse del proceso por si solo (posición actual, por ejemplo). Esto permite crear múltiples iteradores para el mismo contenedor

¿Cómo funciona?

- El cliente solicita al contenedor (IterableConcreto) un iterador.
- Itera mientras el iterador tenga elementos
- El iterador le va retornando de a un elemento







La principal ventaja de esta solución es que permite implementar nuevos tipos de colecciones e iteradores y pasarlos al código existente sin descomponer nada (principio de abierto/cerrado).

Además, se limpia el código del cliente y las colecciones extrayendo algoritmos de recorrido voluminosos y colocándolos en clases independientes (principio de responsabilidad única).

Iterator - Resumen

Intención

- Proporcionar una forma de acceder a los elementos de una colección de objetos de manera secuencial, sin revelar su representación interna.
- Define una interfaz que declara métodos para acceder secuencialmente a la colección.

Ventajas

- Acceso al contenido de una colección de elementos sin exponer su representación interna
- permite varios tipos de recorridos sobre una colección de elementos
- proporciona una interfaz uniforme para recorrer distintos tipos de colecciones (iteración polimórfica)

Inconvenientes

- Aplicar el patrón puede resultar excesivo si tu aplicación funciona únicamente con colecciones sencillas.
- Utilizar un iterador puede ser menos eficiente que recorrer directamente los elementos de algunas colecciones especializadas.

Ejemplo

Supongamos que tenemos una emisora de radio internacional con una lista de canales y el programa cliente desea recorrerlos uno por uno o según el idioma de canal.

Por ejemplo, algunos programas cliente solo están interesados en los canales de inglés y quieren recorrer estos canales y no los demás.

Una alternativa sería dar una colección de canales al cliente y dejar que escriban la lógica para recorrer los canales y decidir si procesarlos o no. Pero con esta solución el cliente tiene que idear la lógica para el recorrido y no podemos asegurarnos de que la lógica del cliente sea correcta.

Este es un ejemplo donde podemos usar el patrón Iterator y proporcionar una iteración basada en el idioma del canal. Debemos asegurarnos de que el programa de cliente pueda acceder a la lista de canales solo a través del iterador.

```java
public class Canal {
    private double frecuencia;
    private Idioma idioma;

    public Canal(double freq, Idioma idioma) {
        this.frecuencia = freq;
        this.idioma = idioma;
    }

    public double getFrecuencia() {
        return frecuencia;
    }

    public Idioma getIdioma() {
        return idioma;
    }

    @Override
    public String toString() {
        return "Frecuencia=" + this.frecuencia + ", Idioma=" + this.idioma;
    }
}
```

```java
public enum Idioma {
    INGLES, FRANCES, ESPANIOL;
}
```

```java
interface IterableCanales extends Iterable<Canal>{
    Iterator<Canal> iterator(Idioma idioma);
}
```

```java
public class Radio implements IterableCanales {

    private List<Canal> canales;

    public Radio() {
        canales = new ArrayList<>();
    }

    public void agregarCanal(Canal c) {
        this.canales.add(c);
    }

    public void eliminarCanal(Canal c) {
        this.canales.remove(c);
    }

    @Override
    public Iterator<Canal> iterator(Idioma idioma) {
        return new IteradorCanales(idioma, this.canales);
    }

    @Override
    public Iterator<Canal> iterator() {
        return new IteradorCanales(this.canales);
    }

    private class IteradorCanales implements Iterator<Canal> {

    … continua en la siguiente …
```

El método que definimos en nuestra interface

Método original (heredado de iterable)

Definimos la clase IteradorCanales privada (dentro de la clase Radio)

```java
private class IteradorCanales implements Iterator<Canal> {
    private Idioma idioma;
    private List<Canal> canales;
    private int position;

    public IteradorCanales(List<Canal> canales) {
        this(null, canales);
    }

    public IteradorCanales(Idioma idioma, List<Canal>
canales) {
        this.idioma = idioma;
        this.canales = canales;
    }

    @Override
    public boolean hasNext() {
        while (position < canales.size()) {
            Canal c = canales.get(position);
            if (c.getIdioma().equals(idioma) || idioma == null) {
                return true;
            } else position++;
        }
        return false;
    }

    @Override
    public Canal next() {
        Canal c = canales.get(position);
        position++;
        return c;
    }

}//fin de la clase IteradorCanales

}//fin de la clase Radio
```

```java
public static void main(String[] args) {
    Radio internacional = Main.crearRadio();

    Iterator<Canal> iteradorBase = internacional.iterator(null);
    while (iteradorBase.hasNext()) {
        Canal c = iteradorBase.next();
        System.out.println(c.toString());
    }

    System.out.println("******");

    Iterator<Canal> canalesEnIngles = internacional.iterator(Idioma.INGLES);
    while (canalesEnIngles.hasNext()) {
        Canal c = canalesEnIngles.next();
        System.out.println(c.toString());
    }
}
```

```java
private static Radio crearRadio() {
    Radio radio = new Radio();
    radio.agregarCanal(new Canal(98.5, Idioma.INGLES));
    radio.agregarCanal(new Canal(99.5, Idioma.ESPANIOL));
    radio.agregarCanal(new Canal(100.5, Idioma.FRANCES));
    radio.agregarCanal(new Canal(101.5, Idioma.INGLES));
    radio.agregarCanal(new Canal(102.5, Idioma.ESPANIOL));
    radio.agregarCanal(new Canal(103.5, Idioma.FRANCES));
    radio.agregarCanal(new Canal(104.5, Idioma.INGLES));
    radio.agregarCanal(new Canal(105.5, Idioma.ESPANIOL));
    radio.agregarCanal(new Canal(106.5, Idioma.FRANCES));
    return radio;
}
```

¿Preguntas?

@MarceloGArmentano

marcelo.armentano@gmail.com